

# temporal-video-translator — Architecture

Version 0.3.10 | generated from code on 2026-07-20

---

## Overview

---

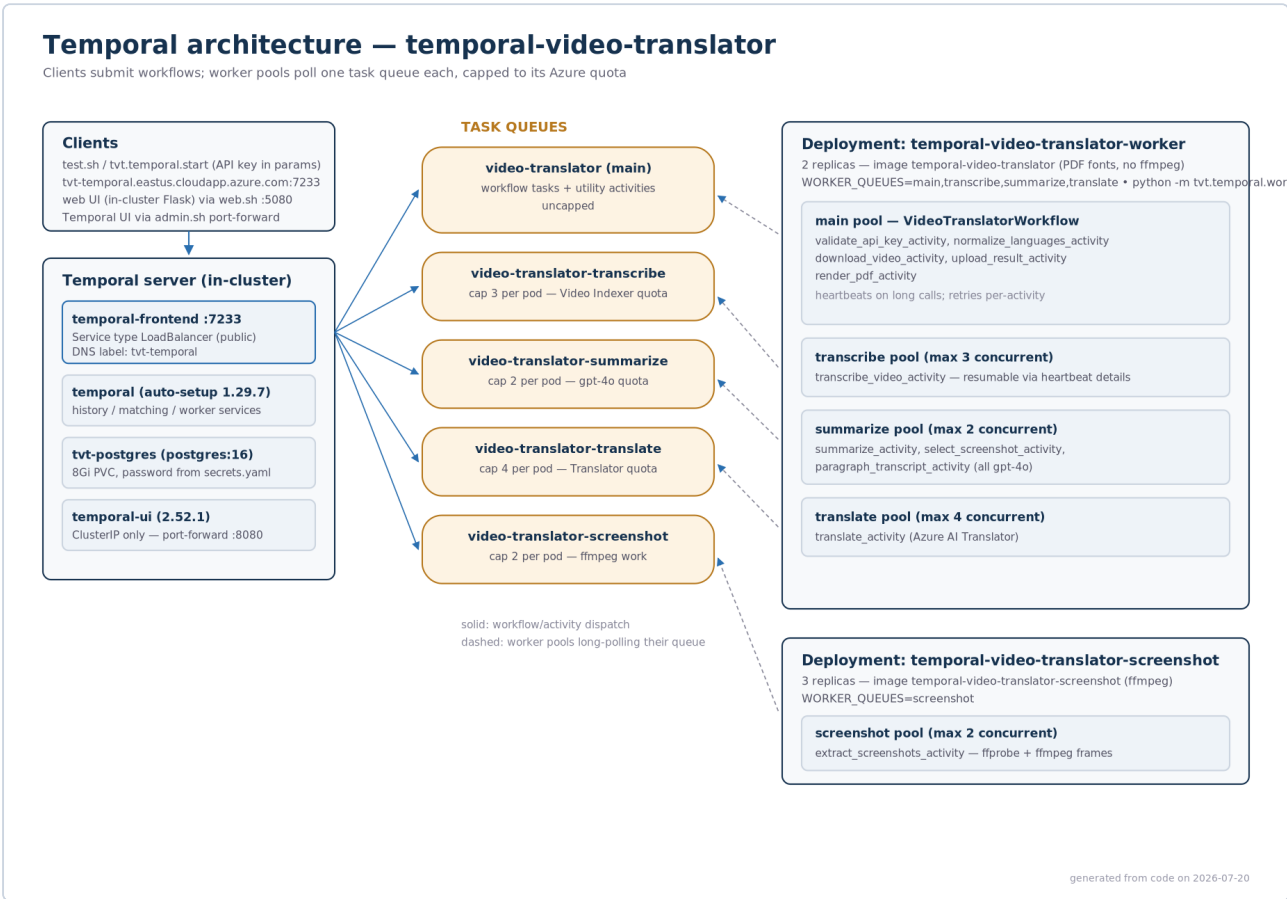
Given a video URL, the system produces a transcript, an approximately 150-word summary, translations into any number of languages, an automatically selected representative screenshot, and a rendered PDF report. A single durable Temporal workflow ( `VideoTranslatorWorkflow` ) orchestrates the pipeline end to end on AKS, delegating the heavy lifting to Azure AI services. Every run returns and publishes three artifacts:

- **Result JSON** — transcript, summary, translations, and artifact URLs, uploaded to the public `json` container as `{workflow-id}-{video}.json`.
- **Screenshot** — the best of three candidate frames (chosen by gpt-4o vision), published to the public `screenshots` container.
- **PDF report** — the result document rendered through an HTML template with WeasyPrint, published to the public `pdf` container.

## Code layout (Python package `tvtt/`)

- `tvtt/temporal/` — workflow, activities, shared dataclasses, worker, CLI starter. The activities module is the composition root wiring Azure into the neutral pieces.
- `tvtt/ai/` — `Summarizer`, `Paragrapher`, `ScreenshotSelector` classes. Provider-neutral: they work on primitives (text, image bytes) against any OpenAI-compatible Responses endpoint, with auth headers injected at construction.
- `tvtt/azure/` — Entra ID credential/token helper and blob storage; `tvtt/azure/ai/` — the Azure-only AI clients (Video Indexer, Translator) and the OpenAI auth provider.
- `tvtt/media/` — video staging, ffmpeg screenshot extraction/promotion, result upload, PDF rendering.
- `tvtt/web/` — Flask UI to submit runs and follow progress.

# Temporal architecture



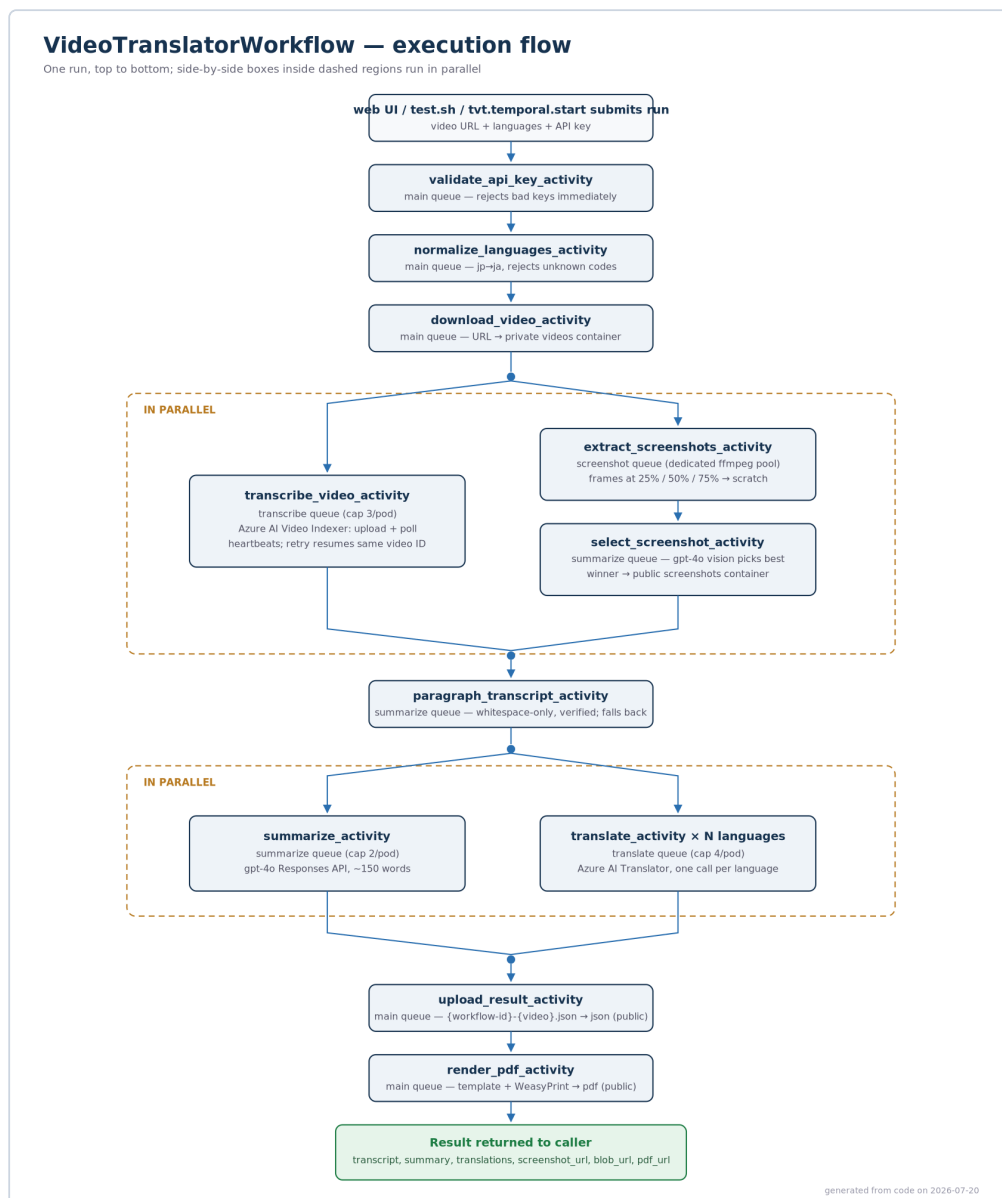
Clients, in-cluster Temporal server, task queues, and the two worker deployments.

Each Azure-bound stage runs on its own task queue with a per-pod concurrency cap sized to that service's quota, so one stage saturating never starves another:

| Task queue                  | Cap / pod | Activities                                                   | Protects            |
|-----------------------------|-----------|--------------------------------------------------------------|---------------------|
| video-translator (main)     | uncapped  | workflow + validate, normalize, download, upload, render_pdf | —                   |
| video-translator-transcribe | 3         | transcribe_video                                             | Video Indexer quota |
| video-translator-summarize  | 2         | summarize, select_screenshot, paragraph_transcript           | gpt-4o quota        |
| video-translator-translate  | 4         | translate                                                    | Translator quota    |
| video-translator-screenshot | 2         | extract_screenshots                                          | ffmpeg CPU          |

One worker process hosts one Temporal Worker per queue; the WORKER\_QUEUES env var selects which pools a deployment runs. The main deployment (2 replicas, PDF/font image) hosts main, transcribe, summarize, translate; a dedicated deployment (3 replicas, slim ffmpeg image) hosts only screenshot, so the compute-intensive ffmpeg pool can be scaled or given special nodes (GPU, high-memory) independently. Caps are per pod — total concurrency is replicas × cap.

## Execution flow



One run, top to bottom; boxes inside dashed regions run in parallel.

- **Fail fast:** the API key is validated first (single attempt, non-retryable) and language codes are normalized ( jp→ja ) with unknown codes rejected — before any expensive work.
- **Parallelism:** transcription runs concurrently with screenshot extraction + selection; later, summarization runs concurrently with all N translations (one activity per language).
- **Heartbeats & resume:** long activities heartbeat every 10 s so dead workers are detected in minutes. The transcribe activity stores the Video Indexer video ID in heartbeat details — a retried attempt resumes polling the same indexing job instead of re-uploading.
- **Retries:** every activity has an explicit RetryPolicy (typically 3-8 attempts) and start-to-close timeout sized to its stage (120 min for indexing, 5-15 min elsewhere).
- **Verified paragraphing:** gpt-4o only returns sentence numbers where paragraphs start; the text is rebuilt locally, verified whitespace-only, and falls back to the raw transcript if the model misbehaves twice.

## Cloud infrastructure

### Cloud infrastructure — temporal-video-translator

Clients (test.sh / tvt.temporal.start, kubectl, helm)  
tvt-temporal.eastus.cloudapp.azure.com:7233 (public LB)

#### Azure resource group: temporal-video-translator (eastus)

##### AKS: tvt-aks (Kubernetes v1.35)

nodepool1: 2 x Standard\_D4d\_v4 • pool2: 1 x Standard\_D4s\_v3

##### temporal-video-translator-worker

2 replicas, main image  
workflow + transcribe/summarize/  
translate/util activity pools

##### temporal-video-translator-screenshot

3 replicas, ffmpeg image  
dedicated screenshot pool only

##### temporal (auto-setup 1.29.7)

Temporal server, all services in one pod  
temporal-frontend Service → public LB

##### tvt-postgres (postgres:16)

Temporal persistence, 8Gi PVC  
(Azure managed disk)

##### temporal-ui (2.52.1)

ClusterIP; admin.sh port-forward

##### Public IP + DNS label

tvt-temporal.eastus.cloudapp.azure.com

##### temporal-video-translator-web (Flask/gunicorn, 1 replica)

submit + progress UI • internal LB (VNet-only) • reached via web.sh port-forward

##### Deliverables (built by Makefile, deployed by helm)

helm chart (workers + web + temporal server + secrets) • secrets.yaml → k8s Secrets  
images: temporal-video-translator (PDF stack) and -screenshot (ffmpeg) • tvt/ package

##### ACR: tvttranslator

tvttranslator.azurecr.io  
temporal-video-translator:{VERSION}  
temporal-video-translator-screenshot  
AKS pulls via kubelet AcrPull role

##### Service principal: tvt-worker

RG roles: Contributor, Cognitive Services  
User, Storage Blob Data Contributor  
AZURE\_CLIENT\_ID/SECRET in k8s Secret;  
Entra ID data-plane auth — no stored keys

##### Storage account: tvttranslations

**videos** staged downloads (private)  
**json** result documents (public)  
**pdf** rendered reports (public)  
**scratchescreenshots** screenshot candidates (private)  
**screenshots** selected best frame (public)  
json + pdf blobs named {workflow-id}-{video}.json/pdf

##### tvt-video-indexer

Azure AI Video Indexer (account 793e...15fc)  
speech-to-text transcription; token via ARM  
generateAccessToken; storage: tvtvideostorage

##### tvt-translator

Azure AI Translator (S1)  
translate v3 + languages endpoint  
Entra bearer + resource-id headers

##### tvt-openai

Azure OpenAI, deployment: gpt-4o  
Responses API: summaries, paragraphing,  
screenshot selection (vision); Entra bearer auth

The temporal-video-translator resource group (eastus): AKS, ACR, storage, AI services, identity.

- **AKS** tvt-aks — nodepool1: 2 x Standard\_D4d\_v4; pool2: 1 x Standard\_D4s\_v3. Runs the two worker deployments, the web UI, and a self-contained Temporal server (auto-setup image + postgres 16 on an 8 Gi PVC + Temporal UI).
- **Entry points** — the Temporal frontend is a public LoadBalancer with DNS label tvt-temporal.eastus.cloudapp.azure.com:7233 ; the workflow itself rejects runs without a valid API key. The web UI sits on an internal (VNet-only) LB reached via web.sh port-forward.
- **ACR** tvttranslator — two image repos built from one Dockerfile: the main image (pango + Noto fonts for PDF) and the screenshot image (ffmpeg, no PDF stack). AKS pulls via the kubelet's AcrPull role.
- **Storage** tvttranslations — private containers videos (staged downloads) and scratchescreenshots (candidates); public containers json, screenshots, pdf (deliverables). Video Indexer keeps its own backing account tvtvideostorage.
- **Identity** — workers run as service principal tvt-worker with RG-scoped roles: Contributor, Cognitive Services User, Storage Blob Data Contributor. All data-plane calls use Microsoft Entra ID bearer tokens via DefaultAzureCredential ; no service keys are fetched or stored. Video Indexer access tokens come from the ARM generateAccessToken API.

## Dependencies

### Azure services

| Service          | Resource            | Used for                                 | Auth                               |
|------------------|---------------------|------------------------------------------|------------------------------------|
| AI Video Indexer | tvt-video-indexer   | speech-to-text transcription             | ARM generateAccessToken            |
| Azure OpenAI     | tvt-openai (gpt-4o) | summary, paragraphing, screenshot vision | Entra bearer                       |
| AI Translator    | tvt-translator (S1) | translations (v3 API)                    | Entra bearer + resource-id headers |
| Blob Storage     | tvttranslations     | staging + published artifacts            | Entra (SDK credential)             |

|           |                           |                  |                 |
|-----------|---------------------------|------------------|-----------------|
| AKS / ACR | tvtt-aks / tvtttranslator | runtime / images | kubelet AcrPull |
|-----------|---------------------------|------------------|-----------------|

## Python (runtime)

temporalio 1.29.0, requests 2.32.3, azure-identity 1.25.3, azure-storage-blob 12.30.0, python-dotenv 1.2.2, jinja2 3.1.6, weasyprint 69.0, flask 3.1.2, gunicorn 23.0.0. Dev adds pytest 8.4.2; the suite covers the AI helpers, translation normalization, screenshot timing, and PDF rendering with all remote calls faked.

## Operational environment

### Configuration & secrets

- Locally: `.env` (see `.env.example`); `DefaultAzureCredential` uses the developer's `az login` (`AZURE_TOKEN_CREDENTIALS=dev` skips the IMDS probe).
- In-cluster: the gitignored `chart-root/secrets.yaml` is passed at install time (`helm -f secrets.yaml`) and lands as Kubernetes Secrets (SP credentials, subscription/account IDs, API key, postgres password); non-secret config is `chart/env`.
- Run gate: callers must present the shared `API_KEY`; the first workflow step rejects mismatches.

### Scaling knobs

- Replicas (`worker.replicas=2`, `screenshotWorker.replicas=3`)  $\times$  per-pod caps (`MAX_CONCURRENT_*`) set total stage concurrency.
- The ffmpeg pool is an isolated deployment — point it at special node pools without touching the main workers.

### Day-to-day commands

| Command                                                                     | Purpose                                                                  |
|-----------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>make test</code>                                                      | run the unit tests                                                       |
| <code>make build / push</code>                                              | build both images, push to ACR (tagged with <code>VERSION</code> )       |
| <code>make deploy</code>                                                    | helm upgrade --install with <code>secrets.yaml</code> , pinned image tag |
| <code>make creds / login</code>                                             | fetch AKS credentials / switch kubectl context                           |
| <code>./test.sh [url] [langs]</code>                                        | submit a run against the AKS deployment                                  |
| <code>python -m tvtt.temporal.worker / python -m tvtt.temporal.start</code> | local worker / submit a run locally                                      |
| <code>./web.sh / ./admin.sh</code>                                          | port-forward the web UI (:5080) / the Temporal UI                        |